

鉴权与授权流程

定义放行路径，这些路径不需要有 token，其余路径都需要走权限过滤器

```
/**
 *放行路径与权限要求，添加自定义权限校验器，不交与Spring管理解决重复经过过滤器的问题
 */
@Bean
public SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity http) {
    http
        .authorizeExchange() .authorizeExchangeSpec {
            //放行路径
            .pathMatchers(
                "/webjars/",
                "/register/**",
                "/user-center/**",
                "/swagger-ui.html",
                "/webjars/**",
                "/swagger-resources/**",
                "/v2/**",
                "/video/**",
                "/searchs/**",
                "/notice/**",
                "/chats/**",
                "/getAllTableData/**",
                "/getComment/**",
                "/getDanmaku/**",
                "/getHistoryVideo/**",
                "/getDetailVideo/**",
                "/getCommendVideo/**",
                "/getFirstPageVideo/**",
                "/getEnsembleVideo/**",
                "/getAllVideoInEnsemble/**",
                "/getUserPrivilege/**",
                "/getPersonalVideo/**",
                "/getRemotelyLikeVideo/**",
                "/getUserInfo/**",
                "/search/**"
            ).permitAll()
            //其他路径需要有role:user权限
            .anyExchange().hasAuthority("role:user")
            .and() ServerHttpSecurity {
                //禁用http基础认证
                .httpBasic().disable()
                //禁用表单认证
                .formLogin().disable()
                //使用自定义过滤器替换默认过滤器
                .addFilterAt(new JwtAuthorizationFilter(), SecurityWebFiltersOrder.AUTHORIZATION)
                //禁止csrf令牌防护
                .csrf().disable();
            }
        }
    return http.build();
}
```

过滤器中从请求头取 token，如果 token 是空返回 401，如果 token 不为空且解析出来是合法未过期 token 且对应权限是配置中指定的 role: user 则放行

授权则是在登录时会返回长短 token 给前端

```

public Result<LoginResponse> passwordLogin(@RequestBody PasswordLoginRequest passwordLoginRequest) {
    if (map.size() > 0) {
        Integer userId = (Integer) map.get(USER_IDENTITY);
        String shortJwt = (String) map.get(SHORT_TOKEN);
        String longJwt = (String) map.get(LONG_TOKEN);
        exchange.getResponse().getHeaders().add(SHORT_TOKEN, shortJwt);
        ResponseCookie cookie = ResponseCookie.from(LONG_TOKEN, longJwt)
            .httpOnly(true)
            .path("/")
            .maxAge(maxAgeSeconds: 60 * 30)
            .build();
        exchange.getResponse().addCookie(cookie);
        return Result.data(new LoginResponse().setStatus(true).setUserId(userId));
    }
    return Result.data(new LoginResponse().setStatus(false));
}

@PostMapping("/phoneNumberLogin")
public Result<LoginResponse> phoneNumberLogin(@RequestBody PhoneNumberLoginRequest phoneNumberLoginRequest, ServerWebExchange exchange) {
    Map<String, Object> map = loginService.phoneNumberLogin(phoneNumberLoginRequest);
    if (map != null) {
        Integer userId = (Integer) map.get(USER_IDENTITY);
        String shortJwt = (String) map.get(SHORT_TOKEN);
        String longJwt = (String) map.get(LONG_TOKEN);
        exchange.getResponse().getHeaders().add(SHORT_TOKEN, shortJwt);
        ResponseCookie cookie = ResponseCookie.from(LONG_TOKEN, longJwt)
            .httpOnly(true)
            .path("/")
            .maxAge(maxAgeSeconds: 60 * 30)
            .build();
        exchange.getResponse().addCookie(cookie);
        log.info("手机号登录的id" + userId);
        return Result.data(new LoginResponse().setStatus(true).setUserId(userId));
    }
    return Result.data(new LoginResponse().setStatus(false));
}

@PostMapping("/mailLogin")
public Result<LoginResponse> mailLogin(@RequestBody MailLoginRequest mailLoginRequest, ServerWebExchange exchange) {
    Map<String, Object> map = loginService.mailLogin(mailLoginRequest);
    if (map != null) {
        Integer userId = (Integer) map.get(USER_IDENTITY);
        String shortJwt = (String) map.get(SHORT_TOKEN);
        String longJwt = (String) map.get(LONG_TOKEN);
        exchange.getResponse().getHeaders().add(SHORT_TOKEN, shortJwt);
        ResponseCookie cookie = ResponseCookie.from(LONG_TOKEN, longJwt)
            .httpOnly(true)
            .path("/")
            .maxAge(maxAgeSeconds: 60 * 30)
            .build();
        exchange.getResponse().addCookie(cookie);
        log.info("邮箱登录的id" + userId);
        return Result.data(new LoginResponse().setStatus(true).setUserId(userId));
    }
    return Result.data(new LoginResponse().setStatus(false));
}

```

双 token 无感刷新

短 token 30min 过期，长 token 7 天过期，短 token 用作鉴权，长 token 用于刷新
长短 token

登录后前端会保持 25min 刷新一次 token，长短 token 都刷新

若无长 token 或长 token 非法或者过期则刷新请求会失败

刷新后会返回两个新的以当前时间为起点来计算过期时间的长短 token

如果用户一直未退出网站就一直会保持刷新，若退出过，长 token 在七天内仍然存在，会在打开页面的那一刻执行一次刷新 token 请求

生成新的长短 token，实现七天内登录一次就能续上 token

```
/**
 *刷新token
 */
@PostMapping("/refreshToken") @Amumu
public Result<Boolean> refreshToken(ServerWebExchange exchange){
    log.info("刷新token");
    String shortToken= exchange.getRequest().getHeaders().getFirst(SHORT_TOKEN);
    log.info(shortToken);
    String longToken=exchange.getRequest().getCookies().getFirst(LONG_TOKEN).getValue();
    exchange.getResponse().getHeaders().set(SHORT_TOKEN, JwtUtil.refreshToken(shortToken, type: 0));
    ResponseCookie cookie = ResponseCookie.from(LONG_TOKEN, JwtUtil.refreshToken(longToken, type: 1))
        .httpOnly(true)
        .path("/")
        .maxAge(LONG_TOKEN_EXPIRATION )
        .build();
    exchange.getResponse().addCookie(cookie);
    return Result.success( data: true);
}
```